

数据库

文件保存数据有以下几个缺点：

- 文件的安全性问题
- 文件不利于数据查询和管理
- 文件不利于存储海量数据
- 文件在程序中控制不方便

为了解决上述问题，专家们设计出更加利于管理数据的软件——数据库，它能更有效的管理数据。数据库可以提供远程服务，即通过远程连接来使用数据库，因此也称为数据库服务器。

什么是数据库？

数据库 (Database) 是按照数据结构来组织、存储和管理数据的仓库。

每个数据库都有一个或多个不同的 API 用于创建，访问，管理，搜索和复制所保存的数据。

数据库分类

- 关系型数据库
 - MySQL、Oracle、SQL Server 等。
- 非关系型数据库
 - MongoDB、Redis、Cassandra 等。
- 分布式数据库
 - Google Spanner、Cassandra 等。
- 内存数据库
 - Redis、Memcached 等。
- 云数据库
 - Amazon RDS、Google Cloud SQL 等。
- 时间序列数据库
 - InfluxDB、TimescaleDB 等。
- 列式存储数据库
 - Cassandra、HBase 等。

现在我们使用关系型数据库管理系统 (RDBMS) 来存储和管理大数据量。所谓的关系型数据库，是建立在关系模型基础上的数据库，借助于集合代数等数学概念和方法来处理数据库中的数据。

RDBMS 即关系数据库管理系统(Relational Database Management System)的特点：

- 数据以表格的形式出现
- 每行为各种记录名称
- 每列为记录名称所对应的数据域
- 许多的行和列组成一张表单
- 若干的表单组成 database

数据库的优点

1. 数据库可以将很多零散的数据，存储成为大量的数据信息，方便用户进行有效的检索和访问，数据更加结构化。同时，数据库可以对数据进行分类以及保存，方便我们快速查询。

2. 可以很好地保证数据的完整性和有效性、不被病毒破坏。因为数据库有避免重复数据的功能，所以当数据储存起来时候，不出出现冗余的情况。
3. 数据储存在数据库中会更加安全，共享交流更加方便。用户要求进入计算机系统时，系统会先进行用户身份的辨识；即使进入系统后，数据库管理系统还会进行存取控制，只允许用户执行合法操作；同时计算机的操作系统也有一套保护措施；数据还可以密码形式存储到数据库中，更加严密。
4. 数据库技术能够提供智能化地分析，产生更多有价值的信息。如今数据挖掘、联机分析等等技术发展特别迅速，其能够从一堆数据中分析出有用的信息，节省更多的时间。
5. 数据库拥有备份的功能，可以让数据更加持久。即使程序发生异常，突然断机这些因素，数据库也能很好的处理这个情况，其拥有恢复数据的机制。如果计算机受到攻击，数据库也能进行备份和恢复。

安装MySQL

下载地址：<https://downloads.mysql.com/archives/community/>

Window

- 双击下载的安装包（例如 `mysql-installer-community-8.0.33.0.msi`），启动安装向导。
- 选择自定义安装（Custom），以便能够选择需要安装的组件。
- 选择 MySQL 服务器和相关组件，确保选择了完整安装或自定义安装，包括服务器、客户端工具等。
- 设置 MySQL 的安装路径和数据存储路径，确保路径中没有中文和特殊字符。

配置MySQL

- 安装完成后，打开 MySQL Installer，选择“MySQL Servers”下的版本（例如 MySQL Server 8.0.33），点击“绿色箭头”移动到右侧框中。
- 点击“Next”进入环境配置界面，设置服务器的类型（开发模式、服务器模式等），选择合适的端口号（默认3306）。
- 设置管理员密码，选择加强版密码加密方式，确保密码强度足够。
- 完成配置后，点击“Execute”开始安装，等待安装完成。

环境变量配置

- 右键点击“此电脑”选择“属性”，点击“高级系统设置”。
- 在系统属性窗口中，点击“环境变量”。
- 在“系统变量”区域找到 Path 变量，点击“编辑”，在打开的窗口中点击“新建”，添加 MySQL 的bin目录路径（例如 `C:\Program Files\MySQL\MySQL Server 8.0\bin`）。
- 确认环境变量设置后，重新打开命令提示符，输入 `mysql --version` 确认环境变量配置成功。

验证安装

- 打开命令提示符（cmd），输入 `mysql -u root -p` 命令，输入之前设置的密码登录 MySQL。
- 输入 `status` 命令查看 MySQL 的版本信息，确认安装成功。

数据库可视化面板

下载地址：<https://dev.mysql.com/downloads/file/?id=528765>

登录和退出数据库

登录

```
mysql -u 用户名 -p
```

退出

```
mysql-> quit
```

数据库-库与表

我们可以将数据库理解为一个图书馆，图书馆中存储了各式各样的书籍，不同类型的书籍都会放在自己对应的区域上，对于书架上的书籍也会有不同属性的分类。

这里我们把库看成图书馆，数据表就是一个一个的区域，数据字段就是书架上书籍的不同的属性。

数据库-库

数据库中的“库”，可以想象成一个大型的综合仓库。这个仓库用于有组织、有规则地存放和管理各种类型的数据。

在“库”里，数据不是杂乱无章地堆积在一起，而是按照一定的数据结构和逻辑关系进行分类和存储。比如，将有关用户信息的数据放在一起，将产品销售的数据放在另一个区域。

“库”的存在使得数据的存储更加集中、规范和安全。它为数据提供了一个统一的管理环境，方便进行数据的添加、删除、修改、查询等操作。同时，它还能够有效地控制数据的访问权限，确保只有授权的人员能够对数据进行相应的操作，保护数据的隐私和完整性。

数据库-表

数据库中，“表”是用于组织和存储数据的基本结构，就好比是一个规整的电子表格。

表由行和列组成。每一行代表一条完整的记录，类似于表格中的一行数据，包含了特定对象或事件的相关信息。每一列则代表一种特定的属性或字段，比如姓名、年龄、地址等。

数据库-字段

“字段”是构成数据库表中列的基本单元，用于定义和描述表中每一列所存储数据的特定属性或特征。

每个字段都有其明确的数据类型，比如整数、字符串、日期、布尔值等，这决定了该字段能够存储的数据形式和范围。例如，一个“年龄”字段可能被定义为整数类型，以存储整数值；而“姓名”字段通常会被定义为字符串类型，用于容纳文字字符。

字段还具有特定的约束条件，例如是否允许为空值、是否唯一等，以保证数据的完整性和准确性。

那么对于图书馆的图书我们要进行查找的时候，管理员会有一套他们自己的方式去找寻图书。对于数据库而言也是如此，会有一套数据库的操作语句来操作数据库。

数据库语句

数据库语句是一种特定的语言表达式，它们遵循严格的语法规则，以准确地传达我们的操作意图。这些语句如同数据库世界的语言，使我们能够对数据库中的数据进行创建、读取、更新和删除等操作

不同的数据库语句具有不同的功能和用途，但它们共同协作，使得数据库能够高效、准确地响应我们的各种需求，实现对数据的有效管理和利用。

数据库的语法规则：

数据库语句；

数据库-常规语句

显示当前 Mysql 的版本

```
SELECT VERSION();
```

显示当前的日期

```
SELECT NOW();
```

显示当前用户

```
SELECT USER();
```

清空数据库终端的显示(调用的是windows 的终端清空命令)

```
mysql->system cls;
```

数据库-库操作语句

显示所有的库

```
SHOW DATABASES;
```

创建一个库

```
CREATE DATABASE 数据库名;
```

创建库的时候指定编码格式

```
CREATE DATABASE 数据库名 character SET 编码;
```

修改库的编码格式

```
ALTER DATABASE 数据库名 character SET 编码;
```

删除一个库

```
DROP DATABASE 数据库名;
```

进入库

```
USE 数据库名;
```

显示当前所在库

```
SELECT DATABASE();
```

数据库-表操作语句

查看当前库中的所有表

```
SHOW TABLES;
```

创建一个数据表

```
CREATE TABLE 表名(字段名1 字段类型1 约束条件, 字段名2 字段类型2 约束条件...);
```

字段的类型

整数类型：

- `TINYINT`：非常小的整数
- `SMALLINT`：较小的整数
- `MEDIUMINT`：中等大小的整数
- `INT` 或 `INTEGER`：标准整数
- `BIGINT`：大整数

字符串类型：

- `CHAR(size)`：固定长度的字符串
- `VARCHAR(size)`：可变长度的字符串
- `TINYTEXT`：小文本字符串
- `TEXT`：文本字符串
- `MEDIUMTEXT`：中等长度文本字符串
- `LONGTEXT`：长文本字符串

浮点数和定点数类型：

- `FLOAT`：单精度浮点数
- `DOUBLE`：双精度浮点数
- `DECIMAL` 或 `NUMERIC`：精确数值，适合用于货币等要求精确度的数据

日期和时间类型

- `DATE`：日期
- `TIME`：时间
- `DATETIME` 或 `TIMESTAMP`：日期和时间组合
- `YEAR`：年份

二进制类型：

- `BINARY(size)`：固定长度的二进制字符串
- `VARBINARY(size)`：可变长度的二进制字符串
- `TINYBLOB`：小二进制对象
- `BLOB`：二进制对象
- `MEDIUMBLOB`：中等大小二进制对象
- `LONGBLOB`：大二进制对象

布尔类型：

- `BOOLEAN` 或 `BOOL`：真/假逻辑变量

枚举和集合类型（主要用于 MySQL/MariaDB）：

- `ENUM(value1,value2,...)`：枚举类型，只能有一个值，且必须是在列表中的一个
- `SET(value1,value2,...)`：集合类型，可以有多达64个成员的集合，多个值之间用逗号隔

约束条件：

非空约束

- `not null` 表示该字段不能为空
- `null` 表示该字段可以为空

主键约束

- 每一个表里面都会存在一个**主键**，主键是这张表的**唯一标识**，可以用来标识这张表的所有数据。
 - 主键不能重复（是唯一的）
 - 主键不能为空（是非空的）
- `primary key` 主键
- `auto_increment` 自动编号

唯一约束

- `unique key` 标识该字段是唯一的。
- 唯一约束可以为空
- 一张表下面可以有多个唯一字段

默认约束

- `default` 创建字段时为默认值

外键约束

- 语法结构：

`foreign key`（子表的数据名） `references` 父表名（父表数据名）

- 建立在两张及以上的表的约束
- 外键列：加入了 `foreign key` 关键词的一列
- 参照列：外键列参照的那一列
- 子表：具有外键列的那个表
- 父表：子表参照的表
- 外键约束：保证约束的字段是唯一的。

查看表结构

两种查询语句

- `show columns from` 数据表名；
- `desc` 数据表名；

作业：

- 创建学生信息表和学生成绩表
 - 学号 主键
 - 身份证 唯一
 - 学号要是外键

删除数据表

```
DROP table 数据表名；
```

修改表

`ALTER` 是一个用于修改表结构和其他数据库对象定义的关键字

修改字段属性

```
alter table 表名 modify 字段名 类型 约束；
```

增加字段(增加列)

```
alter table 表名 add 字段名(列名) 类型(长度) 约束条件；
```

修改字段名

```
alter table 表名 change 旧字段名 新字段名 类型 约束条件；
```

删除字段

```
alter table 表名 drop 字段名；
```

创建约束条件

```
alter table 表名 add constraint 约束条件 字段名(列名)；
```

修改表名

```
rename table 旧表名 to 新表名；
```

修改表的编码

```
alter table 表名 character set 编码;
```

数据操作语句

无非就是对数据库中数据表进行增删改查操作。

增加数据记录

为某个数据表增加一条数据记录

关键字: `insert`

```
INSERT into 表名 (字段名1,字段名2...) values(值1,值2,...);
```

- 没有字段名列表, 那么默认添加全字段。
- 如果填写了字段列表, 那么需要根据字段列表的说明进行数据插入。

查询数据记录

对某个表进行数据的查询操作

关键字: `select`

单表查询

```
SELECT 字段名1,字段名2... FROM 表名;
```

- 可以填写 `*` 来表示全部字段。
- 根据需求在表名后面增加一些查询条件。

Where筛选条件

`where` 用户查询数据的条件筛选。

- `like` 模糊查询, `%` 代表多个字符, `_` 代表一个字符, 例: `like '王%'` 表示开头为王的字符串

```
SELECT * FROM students WHERE name LIKE '王%'; -- 查找姓王开头的名字
```

- `in` 在多个范围指定

```
SELECT * FROM students WHERE age IN (18, 20, 22); -- 查找年龄是 18、20 或 22 的学生
```

- `between and` 区间范围

```
SELECT * FROM students WHERE age BETWEEN 18 AND 20; -- 查找年龄在 18 到 20 之间的学生
```

- `or` 或者 A条件或者B条件满足一个即可


```
SELECT * FROM students WHERE age = 18 OR gender = '女'; -- 查找年龄是 18 或者性别为女的学生
```

- and A条件和B条件同时满足的情况下

```
SELECT * FROM students WHERE age > 18 AND gender = '男'; -- 查找年龄大于 18 且性别为男的学生
```

- order by 排序（根据字段进行排序 asc 正序（默认） desc 倒序）

```
SELECT * FROM students ORDER BY age ASC; -- 按照年龄升序排列
SELECT * FROM students ORDER BY age DESC; -- 按照年龄降序排列
```

- group by 分组（把每一个种类列举一条出来）

```
SELECT gender, COUNT(*) FROM students GROUP BY gender; -- 按照性别分组并统计数量
```

- having 分组筛选（一般情况跟下having会和聚合函数连用）

```
SELECT gender, COUNT(*) FROM students GROUP BY gender HAVING COUNT(*) > 10;
-- 按照性别分组，筛选出数量大于 10 的组
```

- limit 分页 mysql分页

```
SELECT * FROM students LIMIT 10; -- 显示前 10 条记录
SELECT * FROM students LIMIT 5, 10; -- 从第 6 条开始，显示 10 条记录
```

- Limit n; n pagesize -----页显示几条数据
- Limit m,n; m startrow 起始记录 如果m为2 那么 startrow 记录为3,显示n条记录

- distinct 查询去重

```
SELECT DISTINCT class FROM students;
```

别名使用

关键字: as

```
SELECT 表名.字段名 as 别名(重命名) FROM 表名;
```

多表查询

显示两张表的交叉记录（两张表记录的全排列）

内连接

关键字: inner join on 可省略

```
select * from 表1,表2;

select * from 表1 inner join 表2 on 表1.字段名 = 表2.字段名;
```

外连接（可以定义主表）

- 左连接（`left join`）
- 右连接（`right join`）

```
SELECT * FROM 表1 left join 表2 on 表1.字段 = 表2.字段;
```

- 以表1为主表，表2为副表
- 如果表1的记录数多于表2，那么则以表1为主要显示的表，表2的记录数不够则用 `null` 进行填充

子查询

可以在查询语句里面嵌套一个查询语句。

`where` 子查询

```
SELECT * FROM 表名 where name=(SELECT);
```

`in` 子查询

```
SELECT * FROM 表名 where 字段 in(SELECT);
```

`from` 子查询

```
SELECT * FROM (SELECT * FROM 表名)) as a;
```

更新数据

关键字：`update`

```
update 表名 SET 字段名1=新值,字段名2=新值.... WHERE 条件;
```

删除数据

关键字：`delete`

```
delete from 表名;
```

按照需求在表后面增加条件.

作业：

- 自行操作数据操作语句。

事务

在数据库中，事务可以理解为一个逻辑工作单元，它是一组数据库操作的集合，这些操作要么全部成功执行，要么全部不执行，以此来保证数据的一致性和完整性。

比如说，从你的银行账户转账给我，这就不能只扣你的钱而不增加我的钱，或者只增加我的钱而不扣你的钱。这个转账的过程就应该被看作一个事务，要么转账成功，即扣你的钱并且给我加钱都完成；要么转账失败，扣钱和加钱的操作都回滚，就好像没发生过一样。

事务具有四个重要的特性，通常称为 ACID 特性：

- 原子性 (Atomicity)：事务中的所有操作要么全部成功，要么全部失败。
- 一致性 (Consistency)：事务执行的结果必须使数据库从一个合法的状态转变到另一个合法的状态。
- 隔离性 (Isolation)：多个事务并发执行时，它们之间相互隔离，互不干扰。
- 持久性 (Durability)：一旦事务提交，它对数据库的更改就是永久性的，即使系统出现故障也不会丢失。

事务的使用步骤

1. 关闭自动提交
2. 开启事务 (可以省略) `start transaction`
3. 编写 sql 语句，对数据库进行增删改查
4. 结束事务

开启/关闭事务自动提交

关闭了自动提交之后，在操作数据库的时候，本次的 sql 操作是不会同步到数据库上。

```
set autocommit = 0; -- 关闭自动提交
set autocommit = 1; -- 开启自动提交
```

查看事务的开启状态

```
show variables like 'autocommit';
```

事务的操作

其实就是使用sql数据对数据库进行操作。

提交事务

将事件中的 sql 操作进行提交，对数据库进行永久性的更改。

```
commit; -- 提交事务
```

回滚事务

回滚事务，在没有提交事务之前，我们是可以还原操作的。

```
rollback; -- 事务回滚
```

事务保存点

设置一个记录点(还原点)，可以在没提交前，随时返回到记录点。

```
savepoint 记录点名;
```

```
savepoint one_change; -- 设置了一个还原点
```

回滚至某个还原点

```
rollback to 还原点;
```

注意：在还原的时候，所有比当前还原点的时间晚的还原点会全部销毁。

事务并发问题

- **脏读**：有两个事务 T1、T2，T1 更新但未提交数据，被 T2 读取，若 T1 进行回滚，T2 读取的内容就是无效的
- **不可重复读**：有两个事务 T1、T2，T1 读取了一个字段，然后 T2 更新提交了该字段，T1 再次读取同一个字段，值不一样
- **幻读**：有两个事务 T1、T2，T1 读取了一个字段，然后 T2 在该表中插入了新的记录，T1 再次读取同一个表，会读取到多的记录

MySQL 四种事务隔离级别

隔离级别	描述
read uncommitted (读未提交数据)	脏读、不可重复读和幻读的问题都会出现
read committed (读已提交数据)	可以避免脏读，但不可重复读和幻读的问题会出现
repeatable read (可重复读) (MySQL 默认)	在一个事务持续期间时，禁止其他的事务对一个字段的更新提交，也就是说该事务对该字段进行查询的结果不会改变
serializable (串行化)	在一个事务持续期间时，禁止其他的事务对表进行插入，所有并发问题都可以解决，但性能十分低下

更改事务隔离级别

- `session` 当前会话
- `global` 全局

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;  -- 读未提交
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;    -- 读已提交
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;    -- 可重复读
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;       -- 串行化
```

查看事务的隔离级别

```
SELECT @@transaction_isolation;
```

索引

数据库索引就好比一本书的目录。

- 加快数据的查询速度：通过索引，可以快速定位到符合条件的数据所在的位置，而不必全表扫描。
- 提高数据库系统的性能：减少了查询时读取的数据量，从而提高了数据库的响应时间。

添加索引

全文索引: `FULLTEXT`

- 解释: 用于在文本字段中进行快速的全文搜索。它可以对文本内容进行分词处理, 并建立索引, 以便能够高效地查找包含特定关键词或短语的记录。
- 适用场景: 适用于需要在大量文本中搜索关键词或短语的情况, 比如文章内容、评论等。

空间索引: `SPATIAL`

- 解释: 用于处理和优化涉及地理空间数据的查询, 如点、线、多边形等。空间索引可以帮助快速确定与给定空间范围相交或包含在其中的几何对象。
- 适用场景: 在地理信息系统 (GIS)、地图应用、物流路径规划等需要处理地理空间数据的场景中使用。

唯一索引: `UNIQUE`

- 解释: 确保索引列中的值是唯一的, 不允许重复。
- 适用场景: 常用于主键或需要保证唯一性的字段, 如用户 ID 等。

普通索引: 通常创建时不使用特定关键字, 直接使用 `CREATE INDEX` 语句。

- 解释: 可以加快基于该索引列的查询速度, 但允许列中的值重复。
- 适用场景: 对于经常在查询中作为条件的列, 但不要求唯一性时使用。

复合索引: 在创建索引时指定多个列。

- 解释: 基于多个列创建的索引。
- 适用场景: 当查询经常同时涉及多个列时, 创建复合索引可以提高查询性能。

在创建表时添加索引

```
CREATE TABLE 表名(  
    字段名1 .....  
    字段名2 .....  
  
    -- UNIQUE | FULLTEXT | SPATIAL INDEX | KEY 索引名(字段名1 (长度) ASC | DESC) USING  
    索引方法;  
    INDEX (字段名1);  
  
    -- 复合索引  
    INDEX (字段名1, 字段名2);  
)
```

```
-- 创建了一个students的学生表, 并且包含id和name字段, 且为name字段添加了一个唯一索引。  
CREATE TABLE students(id int primary key auto_increment, name varchar(255))  
UNIQUE INDEX unique_name(name));
```

在创建后增加索引

```
/* ALTER TABLE 表名 ADD UNIQUE | FULLTEXT | SPATIAL INDEX | KEY 索引名 (字段名1  
(长度) ASC | DESC) USING 索引方法;  
或  
CREATE UNIQUE | FULLTEXT | SPATIAL INDEX 索引名 ON 表名(字段名) USING 索引方法;  
*/  
  
-- 例:  
-- 一:  
ALTER TABLE 表名 ADD INDEX 索引名(字段);  
-- 二:  
CREATE INDEX 索引名 ON 表名(字段);
```

```
alter table students add index english_index(english);
```

```
create index english_index on students(english);
```

查看索引

```
SHOW INDEX FROM 表名;
```

删除索引

```
DROP INDEX 索引名 ON 表名;
```

```
ALTER TABEL 表名 DROP INDEX 索引名;
```

注意：索引不是越多越好。过多的索引会增加数据插入、更新和删除的开销，因为每次这些操作都需要更新相关的索引。

数据迁移与备份

备份MySQL数据库

- 全量备份

```
$mysqldump -u 用户名 -p 数据库名 > mysql_backup.sql
```

迁移MySQL数据库

```
$mysql -u 用户名 -p 数据库名 < 备份的数据库文件名;
```

C/C++对于MySQL的支持

API 下载网址: <https://dev.mysql.com/downloads/>

- 选择“Include MySQL C Connector”选项，下载MySQL的C连接器。

编译程序连接库：

- `g++ test.cpp -I mysql的头文件目录 -L mysql的库目录 -lmysql的库名`

把压缩包中的 `include` 和 `lib` 目录复制到工程目录下即可。

MySQL 的接口流程

- 初始化 MySQL 库：
 - 初始化 MySQL 库会完成一些必要的内部设置和资源分配，为后续与数据库的交互操作做好准备。同时可以确保库处于一个已知的、正确的初始状态，避免一些潜在的未定义行为和错误。
- 连接 MySQL 数据库
- 执行 SQL 语句
- 获取结果
- 处理结果
- 释放资源
 - 释放结果的资源
- 关闭数据库连接
- 解除 mysql 库初始化

API 基本接口概述

初始化 MySQL 库。

通过调用 `mysql_library_init` 初始化 MySQL 库。

```
int mysql_library_init(int argc, char **argv, char **groups)
/*
    @描述：
        用于初始化MySQL客户端库。
    @argc:
        命令行参数个数
    @argv:
        命令行参数数组
    @groups:
        选项组，用于控制MySQL客户端库的行为

    @返回值： 如果初始化成功就返回0，如果失败就返回非0

    注意：在非多线程环境中，mysql_init() 根据需要自动调用 mysql_library_init()。如果是在
    多线程环境中是要手动去调用这个函数的，比如建立聊天室等，就必须要调用到这个函数，建议写上去比较
    好。在mysql 8.0版本，argc、argv 和 groups 参数未使用，所以写成
    mysql_library_init(0,NULL,NULL)
*/
```

通过调用 `mysql_init` 初始化连接处理程序

```

MYSQL *mysql_init(MYSQL *mysql)
/*
    @描述:
        用于初始化一个MYSQL结构体，返回一个初始化的MYSQL*句柄
    @mysql:
        mysql 一个指向 MYSQL 连接句柄结构的指针。
        如果mysql是NULL，mysql_init() 将分配空间并初始化并返回新的连接句柄。如果mysql不为空，mysql_init对mysql进行初始化，并且同样的指针会被返回。
    @返回值: 一个被始化的MYSQL*句柄，在内存不足的情况下，返回NULL

    注意: 如果 mysql_init() 分配了一个新对象，则在调用 mysql_close() 关闭连接时将其释放。
*/

```

连接到服务器

通过调用 `mysql_real_connect` 连接到服务器

```

MYSQL *mysql_real_connect(MYSQL *mysql, const char *host,
    const char *user,
    const char *passwd,
    const char *db,
    unsigned int port,
    const char *unix_socket,
    unsigned long clientflag);
/*
    @mysql: 一个预先分配和初始化的MYSQL对象指针。该指针用于保存与MySQL服务器的连接相关信息。
    @host: MySQL服务器的主机名或IP地址。
    @user: 连接MySQL服务器所使用的用户名。
    @passwd: 连接MySQL服务器所使用的密码。
    @db: 连接成功后要使用的默认数据库名。
    @port: MySQL服务器的端口号。
    @unix_socket: Unix域套接字路径，用于本地连接Unix系统上的MySQL服务器，一般情况都属设置为nullptr。
    @client_flag: 客户端标志位，用于指定连接选项。
    @return:
        成功，返回一个新的MYSQL对象指针，该对象与第一个参数的值相同，用于后续的MySQL操作。
        失败，返回NULL，并通过调用mysql_errno()和mysql_error()函数获取错误代码和错误信息。
*/

```

发出 SQL 语句

发出 `SQL` 语句。

```

int mysql_query(MYSQL *mysql, const char *query)
/*
    @mysql: 数据库指针
    @query: 数据库操作指令字符串指针
    @return: 成功，返回0；失败，返回非零值，可以通过调用mysql_error()函数获取错误信息。
*/

```


获取结果

通过 `mysql_store_result` 获取结果

```
MYSQL_RES *mysql_store_result(MYSQL *mysql)
/*
  @描述:
    用来获取查询结果的
  @mysql:
    执行了查询结果的数据库句柄
  @return:
    返回结果集
*/
```

处理获取到的结果

通过 `mysql_fetch_row` 处理获取到的结果

```
MYSQL_ROW mysql_fetch_row(MYSQL_RES *result);
/*
  @描述:
    拆分获取的结果
  @result:
    通过mysql_store_result获取到的结果集合
  @return:
    一行一行的数据(一条数据)
*/
```

释放资源

通过 `mysql_free_result` 释放结果的资源

```
mysql_free_result(result); // 释放存放结果的空间
```

关闭MySQL 连接

通过调用 `mysql_close` 关闭与 `MySQL` 服务器的连接。

```
void mysql_close(MYSQL *connection);
/*
  @connection: 指向一个已经打开的MYSQL对象的指针。该对象代表与MySQL服务器的连接。
  注意:
    在调用mysql_close之前, 确保已经完成了与数据库的所有操作, 包括查询和事务等。
    关闭连接后, 将无法再使用该连接对象执行任何操作, 除非重新使用mysql_real_connect建立新的连接。
    对于多个线程同时共享一个连接对象的情况, 应该确保在多个线程之间正确地协调和同步连接的打开和关闭操作, 以避免出现竞态条件和意外错误。
*/
```

解除初始化

```
mysql_library_end(); // 终止库
```

错误信息

```
const char *mysql_error();
/*
    @描述:
        发送错误了之后，可以调用该函数，然后返回错误信息
    @return:
        错误信息的字符串形式
*/
```

API 基本数据结构参考

名字	描述
MYSQL	这个结构代表一个数据连接的句柄，它几乎用于所有 MySQL 函数。
MYSQL_RES	这个结构体代表一个询问语句的返回值（SELECT，SHOW，DESCRIBE，EXPLAIN），查询语句返回的信息叫做结果集
MYSQL_ROW	这是一行数据的“类型安全”表示。，可以把它看成一个字符串集（但是如果他包含二进制数据的话，就不能这样做，因为很多数据内部都包含Null字节），行是通过 mysql_fetch_row 获得的。

API 基本函数参考

名字	描述	是否弃用
<code>mysql_affected_rows()</code>	上次 <code>更新</code> 、删除或插入语句更改/删除/插入的行数	
<code>mysql_autocommit()</code>	设置自动提交模式	
<code>mysql_bind_param()</code>	为执行的下一条语句定义查询属性	
<code>mysql_change_user()</code>	在打开的连接上更改用户和数据库	
<code>mysql_character_set_name()</code>	当前连接的默认字符集名称	
<code>mysql_close()</code>	关闭与服务器的连接	
<code>mysql_commit()</code>	提交事务	
<code>mysql_connect()</code>	连接到 MySQL 服务器，现已改用 <code>mysql_real_connect()</code>	是
<code>mysql_create_db()</code>	创建数据库，改用 <code>mysql_real_query</code> 或 <code>mysql_query</code> 发出 SQL <code>CREATE DATABASE</code> 语句	是
<code>mysql_data_seek()</code>	查找查询结果集中的任意行号	
<code>mysql_debug()</code>	使用给定字符串执行 <code>DEBUG_PUSH</code>	
<code>mysql_drop_db()</code>	删除数据库，改用 <code>mysql_real_query</code> 或 <code>mysql_query</code> 发出 SQL <code>DROP DATABASE</code> 语句	是
<code>mysql_dump_debug_info()</code>	导致服务器将调试信息写入错误日志	
<code>mysql_eof()</code>	确定是否已读取结果集的最后一行	是
<code>mysql_errno()</code>	最近调用的 MySQL 函数的错误号	
<code>mysql_error()</code>	最近调用的 MySQL 函数的错误消息	
<code>mysql_escape_string()</code>	转义字符串中用于 SQL 语句的特殊字符	
<code>mysql_fetch_field()</code>	下一个表字段的类型	
<code>mysql_fetch_field_direct()</code>	给定字段编号的表字段类型	
<code>mysql_fetch_fields()</code>	返回所有字段结构的数组	
<code>mysql_fetch_lengths()</code>	返回当前行中所有列的长度	
<code>mysql_fetch_row()</code>	提取下一个结果集行	
<code>mysql_field_count()</code>	最新语句的结果列数	
<code>mysql_field_seek()</code>	查找结果集行中的列	

名字	描述	是否弃用
<code>mysql_field_tell()</code>	上次 <code>mysql_fetch_field</code> 调用的字段位置	
<code>mysql_free_result()</code>	释放结果集内存	
<code>mysql_free_ssl_session_data()</code>	释放上次 <code>mysql_get_ssl_session_data</code> 调用的会话数据句柄	
<code>mysql_get_character_set_info()</code>	有关默认字符集的信息	
<code>mysql_get_client_info()</code>	客户端版本（字符串）	
<code>mysql_get_client_version()</code>	客户端版本（整数）	
<code>mysql_get_host_info()</code>	有关连接的信息	
<code>mysql_get_option()</code>	<code>mysql_options</code> 选项的值	
<code>mysql_get_proto_info()</code>	连接使用的协议版本	
<code>mysql_get_server_info()</code>	服务器版本号（字符串）	
<code>mysql_get_server_version()</code>	服务器版本号（整数）	
<code>mysql_get_ssl_cipher()</code>	当前 SSL 密码	
<code>mysql_get_ssl_session_data()</code>	返回启用 SSL 的连接的会话数据	
<code>mysql_get_ssl_session_reused()</code>	会话是否重复使用	
<code>mysql_hex_string()</code>	以十六进制格式对字符串进行编码	
<code>mysql_info()</code>	有关最近执行的语句的信息	
<code>mysql_init()</code>	获取或初始化结构 <code>MYSQL</code>	
<code>mysql_insert_id()</code>	为列生成的 ID 以前的声明 <code>AUTO_INCREMENT</code>	
<code>mysql_kill()</code>	终止线程	是
<code>mysql_library_end()</code>	完成 MySQL C API 库	
<code>mysql_library_init()</code>	初始化 MySQL C API 库	
<code>mysql_list_dbs()</code>	返回与正则表达式匹配的数据库名称	
<code>mysql_list_fields()</code>	返回与正则表达式匹配的字段名称	
<code>mysql_list_processes()</code>	当前服务器线程列表	
<code>mysql_list_tables()</code>	返回与正则表达式匹配的表名	
<code>mysql_more_results()</code>	检查是否存在更多结果	
<code>mysql_next_result()</code>	在多结果执行中返回/启动下一个结果	

名字	描述	是否弃用
<code>mysql_num_fields()</code>	结果集中的列数	
<code>mysql_num_rows()</code>	结果集中的行数	
<code>mysql_options()</code>	连接前设置选项	
<code>mysql_options4()</code>	连接前设置选项	
<code>mysql_ping()</code>	Ping 服务器	
<code>mysql_query()</code>	执行语句	
<code>mysql_real_connect()</code>	连接到 MySQL 服务器	
<code>mysql_real_connect_dns_srv()</code>	使用 DNS SRV 记录连接到 MySQL 服务器	
<code>mysql_real_escape_string()</code>	对语句字符串中的特殊字符进行编码	
<code>mysql_real_escape_string_quote()</code>	对语句字符串中的特殊字符进行编码，以引用上下文	
<code>mysql_real_query()</code>	执行语句	
<code>mysql_refresh()</code>	刷新或重置表和缓存	
<code>mysql_reload()</code>	重新加载授权表	是
<code>mysql_reset_connection()</code>	重置连接以清除会话状态	
<code>mysql_reset_server_public_key()</code>	从客户端库中清除缓存的 RSA 公钥	
<code>mysql_result_metadata()</code>	结果集是否具有元数据	
<code>mysql_rollback()</code>	回滚事务	
<code>mysql_row_seek()</code>	查找结果集中的行偏移量	
<code>mysql_row_tell()</code>	结果集行中的当前位置	
<code>mysql_select_db()</code>	选择数据库	
<code>mysql_server_end()</code>	完成 MySQL C API 库	
<code>mysql_server_init()</code>	初始化 MySQL C API 库	
<code>mysql_session_track_get_first()</code>	会话状态更改信息的第一部分	
<code>mysql_session_track_get_next()</code>	会话状态更改信息的下一部分	
<code>mysql_set_character_set()</code>	设置当前连接默认字符集	
<code>mysql_set_local_infile_default()</code> 处理程序回调到默认值		

名字	描述	是否弃用
<code>mysql_set_local_infile_handler()</code> 处理程序回调		
<code>mysql_set_server_option()</code>	设置当前连接的选项	
<code>mysql_shutdown()</code>	关闭 MySQL 服务器	
<code>mysql_sqlstate()</code>	最近调用的 MySQL 函数的 SQLSTATE 值	
<code>mysql_ssl_set()</code>	准备与服务器建立 SSL 连接	
<code>mysql_stat()</code>	服务器状态	
<code>mysql_store_result()</code>	检索和存储整个结果集	
<code>mysql_thread_id()</code>	当前线程 ID	
<code>mysql_use_result()</code>	启动逐行结果集检索	
<code>mysql_warning_count()</code>	上一个语句的警告计数	

示例代码

```
#include <iostream>
#include <mysql.h>

int main()
{
    // 初始化数据库链接库
    mysql_library_init(0, NULL, NULL);
    // 初始化数据库
    MYSQL *mysql = mysql_init(NULL);
    // 连接数据库
    if(mysql_real_connect(mysql, "localhost", "root", "123456", NULL, 0, NULL, 0) ==
    NULL)
        std::cout << "Mysql connect failed:" << mysql_error(mysql) << std::endl;
    else
        std::cout << "Mysql connect success" << std::endl;
    // 操作数据库，执行数据库语句
    std::string sql = "show databases";
    if(mysql_real_query(mysql, sql.c_str(), sql.length()) != 0)
        std::cout << "Sql query failed:" << mysql_error(mysql) << std::endl;
    else
    {
        // 获取结果集
        MYSQL_RES *result = mysql_store_result(mysql);
        // 处理结果集
        MYSQL_ROW row = NULL;
        while(1)
        {
            row = mysql_fetch_row(result);
            if(row == NULL)break;
        }
    }
}
```

```
        // 打印结果集
        std::cout << row[0] << std::endl;
    }
    // 释放结果集
    mysql_free_result(result);
}
// 关闭数据库
mysql_close(mysql);
// 释放数据库链接库
mysql_library_end();
return 0;
}
```

作业:

- 练习一下 `Mysql C` 的 API 使用。